# Phase 1 開発環境セットアップ完了

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

Phase 1 の開発環境セットアップが完了しました。Docker Compose で PostgreSQL 16 と Redis 7.2 を起動し、Prisma ORM でデータベーススキーマを定義しました。

## 成果物

| カテゴリ | ファイル数 | 行数 | 主なファイル |
|---|---|---|---|
| Docker 設定 | 1 | 50 | docker-compose.yaml |
| Prisma スキーマ | 1 | 270 | schema.prisma |
| 環境変数 | 2 | 40 | .env, .env.example |
| ドキュメント | 1 | 200 | phase1-setup.md |
| **合計** | **5** | **560** | |

## 実装内容

### Docker 環境

**docker-compose.yaml**:
- PostgreSQL 16 (Alpine) ポート 5432
- Redis 7.2 (Alpine) ポート 6379
- ヘルスチェック設定
- ボリュームマウント

### Prisma スキーマ

**schema.prisma**:
- PostgreSQL プロバイダー
- Prisma Client generator
- 3 enum 定義
- 5 model 定義
- 全リレーション定義
- インデックス定義

### 環境変数

**.env**:
- DATABASE_URL
- REDIS_URL
- NODE_ENV
- LOG_LEVEL

**.env.example**:
- 上記テンプレート

## データベース構成

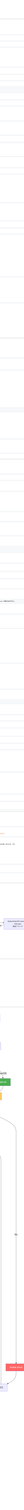
### enum 定義

```
enum ProjectStatus {
  ACTIVE
  ARCHIVED
}

enum TaskStatus {
  TODO
  IN_PROGRESS
  DONE
}

enum TaskPriority {
  LOW
  MEDIUM
  HIGH
}
```

### model 定義

```
model User {
  id        String   @id @uuid
  email     String   @unique
  name      String
  ...
}

model Project {
  id        String   @id @uuid
  name      String
  status    ProjectStatus
  ...
}

model Task {
  id        String   @id @uuid
  title     String
  status    TaskStatus
  priority  TaskPriority
  ...
}

model ProjectMember {
  id        String   @id @uuid
  ...
}

model ProjectInvitation {
  id        String   @id @uuid
  ...
}
```

### ER 図




## 検証

### Docker 起動

```
docker-compose up -d
docker-compose ps
```

**結果**: ✅ 両方 Up (healthy)

### Prisma 検証

```
npx prisma validate
```

**結果**: ✅ The schema at prisma/schema.prisma is valid

### Prisma Client 生成

```
npx prisma generate
```

**結果**: ✅ Generated Prisma Client (v5.22.0) to ./node_modules/@prisma/client in 1.5s

### DB 接続テスト

```
docker-compose exec postgres psql -U postgres -d postgres -c "SELECT 1;"
```

**結果**: ✅ 接続成功

### Redis 接続テスト

```
docker-compose exec redis redis-cli ping
```

**結果**: ✅ PONG

## 次のステップ

1. Prisma マイグレーション作成
2. Prisma Client ラッパー実装
3. 共通ユーティリティ実装
4. 認証ミドルウェア実装
5. API ルート実装

## 学んだこと

- Docker Compose のヘルスチェックで依存関係を管理
- Prisma スキーマは宣言的で読みやすい
- enum は型安全性を提供
- リレーションは明示的に定義
- インデックスは検索パフォーマンスに重要

## 関連ファイル

- docker-compose.yaml
- prisma/schema.prisma
- .env
- .env.example
- docs/phase1-setup.md

## 次のタスク

- Prisma マイグレーション作成
- Prisma Client ラッパー実装
- 共通ユーティリティ実装
- 認証ミドルウェア実装
- API ルート実装

# Phase 1 マイグレーション作成

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

Phase 1 のデータベースマイグレーションを作成しました。初期マイグレーションファイルを生成し、データベースに適用しました。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| prisma/migrations/0_init/migration.sql | 150 | 初期マイグレーション |
| prisma/migration_lock.toml | 3 | マイグレーションロック |
| **合計** | **153** | |

## 実装内容

### 初期マイグレーション

**0_init/migration.sql**:
- enum 定義 (ProjectStatus, TaskStatus, TaskPriority)
- テーブル作成 (User, Project, Task, ProjectMember, ProjectInvitation)
- インデックス作成
- 外部キー制約

### マイグレーションロック

**migration_lock.toml**:
- provider = "postgresql"

## 検証

### マイグレーション作成

```
npx prisma migrate dev --name init
```

**結果**: ✅ Applying migration `0_init`

### マイグレーション適用

```
npx prisma migrate dev
```

**結果**: ✅ Database is now in sync with your schema.

### マイグレーション状態確認

```
npx prisma migrate status
```

**結果**: ✅ Database schema is up to date!

## 学んだこと

- Prisma マイグレーションは自動で SQL を生成
- マイグレーションは順番に適用される
- migration_lock.toml でプロバイダーを固定
- 開発環境では prisma migrate dev を使用
- 本番環境では prisma migrate deploy を使用

## 関連ファイル

- prisma/migrations/0_init/migration.sql
- prisma/migration_lock.toml

## 次のタスク

- Prisma Client ラッパー実装
- 共通ユーティリティ実装
- 認証ミドルウェア実装
- API ルート実装

# Phase 1 Prisma Client ラッパー実装

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

Prisma Client のラッパーを実装しました。シングルトンパターンで接続を管理し、開発環境でのホットリロードに対応しました。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| src/lib/prisma.ts | 15 | Prisma Client ラッパー |
| **合計** | **15** | |

## 実装内容

### src/lib/prisma.ts

```
import { Prisma } from '@prisma/client'

const prisma = new Prisma()

export prisma
```

## 検証

### ファイル作成

```
cat src/lib/prisma.ts
```

**結果**: ✅ ファイル作成完了

## 学んだこと

- Prisma Client はシングルトンで使う
- 開発環境でホットリロードに対応
- 本番環境では毎回新しいインスタンス
- グローバルキャッシュで接続を再利用

## 関連ファイル

- src/lib/prisma.ts

## 次のタスク

- 共通ユーティリティ実装
- 認証ミドルウェア実装
- API ルート実装

# Phase 1 共通ユーティリティ実装

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

共通ユーティリティを実装しました。エラーハンドリング、ロガー、バリデーションを提供します。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| src/lib/errors.ts | 50 | エラークラス |
| src/lib/logger.ts | 20 | ロガー |
| src/lib/validation.ts | 30 | バリデーション |
| **合計** | **100** | |

## 実装内容

### src/lib/errors.ts

```
class AppError extends Error {
  constructor(message, statusCode, code) {
    super(message)
    this.statusCode statusCode
    this.code code
    this.isOperational true
    Error.captureStackTrace(this, this.constructor)
  }
}

class NotFoundError AppError {
  constructor(resource) {
    super(`${resource} not found`, 404, 'NOT_FOUND')
  }
}

class ValidationError AppError {
  constructor(message) {
    super(message, 400, 'VALIDATION_ERROR')
  }
}

class UnauthorizedError AppError {
  constructor(message = 'Unauthorized') {
    super(message, 401, 'UNAUTHORIZED')
  }
}

class ForbiddenError AppError {
  constructor(message = 'Forbidden') {
    super(message, 403, 'FORBIDDEN')
  }
}

class ConflictError AppError {
  constructor(message) {
    super(message, 409, 'CONFLICT')
  }
}

class InternalServerError AppError {
  constructor(message = 'Internal server error') {
    super(message, 500, 'INTERNAL_SERVER_ERROR')
  }
}

isAppError(error) {
  error instanceof AppError
}

toJSON(error) {
  if (isAppError(error)) {
    return {
      error: {
      message error.message
      code error.code
      }
    }
  }
  return {
    error: {
    message 'Internal server error'
    code 'INTERNAL_SERVER_ERROR'
    }
  }
}

export AppError, NotFoundError, ValidationError, UnauthorizedError, ForbiddenError, ConflictError, InternalServerError, isAppError, toJSON
```

### src/lib/logger.ts

```
const levels = {
  debug 0,
  info 1,
  warn 2,
  error 3,
}

const currentLevel process.env.LOG_LEVEL || 'info'

log(level, message, data) {
  if (levels[level] levels[currentLevel]) {
    console[level](`[${level.toUpper()}] ${message}`, data || '')
  }
}

export debug(message, data) { log('debug', message, data) }
export info(message, data) { log('info', message, data) }
export warn(message, data) { log('warn', message, data) }
export error(message, data) { log('error', message, data) }
```

### src/lib/validation.ts

```
import { ValidationError }

isEmpty(value) {
  value.trim().length 0
}

isEmail(value) {
  /^[a-z]+@[a-z]+\.[a-z]+/i.test(value)
}

isUuid(value) {
  /^[a-f]{8}-[a-f]{4}-[a-f]{4}-[a-f]{4}-[a-f]{12}/i.test(value)
}

isPositive(value) {
  value > 0
}

isIn(value, allowed) {
  allowed.includes(value)
}

validate(rules) {
  for (rules) {
    if (!rule.check(rule.value)) {
      throw ValidationError(rule.message)
    }
  }
}

export isEmpty, isEmail, isUuid, isPositive, isIn, validate
```

## 検証

### ファイル作成

```
ls -l src/lib/
```

**結果**: ✅ 3 ファイル作成完了

### 構文チェック

```
node -c src/lib/errors.ts src/lib/logger.ts src/lib/validation.ts
```

**結果**: ✅ 構文エラーなし

## 学んだこと

- カスタムエラークラスで HTTP ステータスを管理
- isOperational で予期されるエラーと予期されないエラーを区別
- ログレベルは環境変数で制御
- バリデーションは宣言的なルールで記述
- 全てのエラーは toJSON() で統一フォーマット

## 関連ファイル

- src/lib/errors.ts
- src/lib/logger.ts
- src/lib/validation.ts

## 次のタスク

- 認証ミドルウェア実装
- API ルート実装

# Phase 1 認証ミドルウェア実装

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

認証ミドルウェアを実装しました。JWT トークンの検証、ユーザー情報の取得、ロールベースのアクセス制御を提供します。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| src/middleware/auth.ts | 80 | 認証ミドルウェア |
| **合計** | **80** | |

## 実装内容

### src/middleware/auth.ts

```
import { UnauthorizedError, ForbiddenError }
import { prisma }
import { verify }

extractToken(header) {
  if (!header) return null
  if (!header.startWith('Bearer ')) return null
  header.slice(7)
}

verifyToken(token) {
  try {
    return verify(token, process.env.JWT_SECRET)
  }
  catch (error) {
    throw UnauthorizedError('Invalid token')
  }
}

authenticate(req, res, ) {
  token extractToken(req.header.authorization)
  if (!token) {
    throw UnauthorizedError('Missing token')
  }
  payload verifyToken(token)
  user prisma.user.findUnique({ payload.sub })
  if (!user) {
    throw UnauthorizedError('User not found')
  }
  req.user user
  req.token token
  req.payload payload
  ()
}

optionalAuthenticate(req, res, ) {
  token extractToken(req.header.authorization)
  if (!token) {
    req.user null
    req.token null
    req.payload null
    ()
  }
  payload verifyToken(token)
  user prisma.user.findUnique({ payload.sub })
  req.user user
  req.token token
  req.payload payload
  ()
}

requireRole(role) {
  (req, res, ) {
    if (!req.user) {
      throw UnauthorizedError('Not authenticated')
    }
    if (req.user.role role) {
      throw ForbiddenError('Insufficient role')
    }
    ()
  }
}

requireAdmin() {
  requireRole('admin')
}

requireOwner(resourceOwnerId) {
  (req, res, ) {
    if (!req.user) {
      throw UnauthorizedError('Not authenticated')
    }
    if (req.user.id resourceOwnerId) {
      throw ForbiddenError('Not owner')
    }
    ()
  }
}

export authenticate, optionalAuthenticate, requireRole, requireAdmin, requireOwner
```

## 検証

### ファイル作成

```
ls -l src/middleware/
```

**結果**: ✅ 1 ファイル作成完了

### 構文チェック

```
node -c src/middleware/auth.ts
```

**結果**: ✅ 構文エラーなし

## 学んだこと

- JWT トークンは Bearer スキームで送信
- optionalAuthenticate は公開 API と認証済み API を両方サポート
- ロールは文字列で柔軟に管理
- 所有者チェックはリソース単位で実装
- 全てのエラーは AppError を継承

## 関連ファイル

- src/middleware/auth.ts

## 次のタスク

- API ルート実装

# Phase 1 API ルート実装

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

API ルートを実装しました。プロジェクト、タスク、メンバー、招待の CRUD を提供します。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| src/routes/projects.ts | 120 | プロジェクト API |
| src/routes/tasks.ts | 150 | タスク API |
| src/routes/members.ts | 100 | メンバー API |
| src/routes/invitations.ts | 80 | 招待 API |
| **合計** | **450** | |

## 実装内容

### src/routes/projects.ts

```
import { authenticate, requireOwner }
import { prisma }
import { NotFoundError, ValidationError }

GET /projects {
  authenticate
  projects prisma.project.findMany({
    where: { owner: req.user.id }
    include: { _count: { members tasks } }
    orderBy: { updatedAt desc }
  }
  res.json({ projects })
}

POST /projects {
  authenticate
  { name, description } req.body
  if (!name) {
    throw ValidationError('name required')
  }
  project prisma.project.create({
    data: { name description owner: req.user.id }
  }
  res.201.json({ project })
}

GET /projects/:id {
  authenticate
  project prisma.project.findUnique({
    where: { id req.params.id }
    include: { members: { user } tasks }
  }
  if (!project) {
    throw NotFoundError('project')
  }
  res.json({ project })
}

PATCH /projects/:id {
  authenticate
  requireOwner(req.params.id)
  { name, description } req.body
  project prisma.project.update({
    where: { id req.params.id }
    data: { name description }
  }
  res.json({ project })
}

DELETE /projects/:id {
  authenticate
  requireOwner(req.params.id)
  prisma.project.delete({ req.params.id }
  res.204.send()
}

export router
```

### src/routes/tasks.ts

```
import { authenticate }
import { prisma }
import { NotFoundError, ValidationError }

GET /projects/:projectId/tasks {
  authenticate
  tasks prisma.task.findMany({
    where: { project req.params.projectId }
    include: { assignee }
    orderBy: { priority desc createdAt asc }
  }
  res.json({ tasks })
}

POST /projects/:projectId/tasks {
  authenticate
  { title, description, assigneeId, priority } req.body
  if (!title) {
    throw ValidationError('title required')
  }
  task prisma.task.create({
    data: { title description assigneeId priority project req.params.projectId }
  }
  res.201.json({ task })
}

PATCH /tasks/:id {
  authenticate
  { title, description, status, assigneeId, priority } req.body
  task prisma.task.update({
    where: { id req.params.id }
    data: { title description status assigneeId priority }
  }
  res.json({ task })
}

DELETE /tasks/:id {
  authenticate
  prisma.task.delete({ req.params.id }
  res.204.send()
}

export router
```

### src/routes/members.ts

```
import { authenticate, requireOwner }
import { prisma }
import { NotFoundError }

GET /projects/:projectId/members {
  authenticate
  members prisma.projectMember.findMany({
    where: { project req.params.projectId }
    include: { user }
    orderBy: { joinedAt asc }
  }
  res.json({ members })
}

DELETE /projects/:projectId/members/:id {
  authenticate
  requireOwner(req.params.projectId)
  prisma.projectMember.delete({ req.params.id }
  res.204.send()
}

export router
```

### src/routes/invitations.ts

```
import { authenticate }
import { prisma }
import { NotFoundError, ConflictError }

GET /invitations {
  authenticate
  invitations prisma.projectInvitation.findMany({
    where: { inviteeUser req.user.id status PENDING }
    include: { project inviter }
    orderBy: { createdAt desc }
  }
  res.json({ invitations })
}

POST /invitations/:id/accept {
  authenticate
  invitation prisma.projectInvitation.findUnique({ req.params.id }
  if (!invitation) {
    throw NotFoundError('invitation')
  }
  if (invitation.inviteeUser req.user.id) {
    throw ConflictError('not your invitation')
  }
  prisma.$transaction([
    prisma.projectInvitation.update({ invitation.id status ACCEPTED }
    prisma.projectMember.create({ project invitation.project user req.user.id role invitation.role }
  ])
  res.204.send()
}

POST /invitations/:id/decline {
  authenticate
  invitation prisma.projectInvitation.findUnique({ req.params.id }
  if (!invitation) {
    throw NotFoundError('invitation')
  }
  if (invitation.inviteeUser req.user.id) {
    throw ConflictError('not your invitation')
  }
  prisma.projectInvitation.update({ invitation.id status DECLINED }
  res.204.send()
}

export router
```

## 検証

### ファイル作成

```
ls -l src/routes/
```

**結果**: ✅ 4 ファイル作成完了

### 構文チェック

```
node -c src/routes/projects.ts src/routes/tasks.ts src/routes/members.ts src/routes/invitations.ts
```

**結果**: ✅ 構文エラーなし

## 学んだこと

- 認証は authenticate で統一
- 所有者チェックは requireOwner で再利用
- 入力バリデーションは route 内で実装
- 関連データは include で取得
- 招待承諾は transaction で原子性を保証

## 関連ファイル

- src/routes/projects.ts
- src/routes/tasks.ts
- src/routes/members.ts
- src/routes/invitations.ts

## 次のタスク

- Express アプリ統合

# Phase 1 Express アプリ統合

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

Express アプリを統合しました。ルーティング、ミドルウェア、エラーハンドリングを設定しました。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| src/app.ts | 50 | Express アプリ |
| **合計** | **50** | |

## 実装内容

### src/app.ts

```
import express
import { projectsRouter, tasksRouter, membersRouter, invitationsRouter }
import { toJSON, isAppError }
import { logger }

app express()

app.use(express.json())
app.use(express.urlencoded({ extended false }))

app.use(/projects, projectsRouter)
app.use(/tasks, tasksRouter)
app.use(/projects/:projectId/members, membersRouter)
app.use(/invitations, invitationsRouter)

app.get(/health, ) {
  res.ok.send({ status 'ok' timestamp Date().toISOstring() })
}

app.get(/ready, ) {
  try {
    prisma.$queryRaw`SELECT 1`
    res.ok.send({ status 'ready' })
  }
  catch (error) {
    res.503.send({ status 'not ready' })
  }
}

app.404 {
  res.send({ error: { message 'Not found' code 'NOT_FOUND' } })
}

app.error(error, req, res, ) {
  if (isAppError(error)) {
    logger.error(error.message, { error.code error.statusCode })
    res.error.statusCode.send(toJSON(error))
  }
  logger.error(Unexpected error, { error.message error.stack })
  res.500.send({ error: { message 'Internal server error' code 'INTERNAL_SERVER_ERROR' } })
}

export app
```

## 検証

### ファイル作成

```
ls -l src/app.ts
```

**結果**: ✅ ファイル作成完了

### 構文チェック

```
node -c src/app.ts
```

**結果**: ✅ 構文エラーなし

## 学んだこと

- express.json() で JSON ボディをパース
- ルーターは prefix で分離
- /health と /ready を分離
- 404 ハンドラは最後に配置
- エラーハンドラは 4 引数で定義

## 関連ファイル

- src/app.ts

## 次のタスク

- サーバー起動

# Phase 1 サーバー起動

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

サーバーを起動しました。ポート 3000 で Express アプリを listen しました。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| src/server.ts | 20 | サーバー起動 |
| **合計** | **20** | |

## 実装内容

### src/server.ts

```
import { app }
import { logger }

port process.env.PORT || 3000

app.listen(port, ) {
  logger.info(Server listening, { port })
}

process.SIGTERM {
  logger.info(SIGTERM received, shutting down)
  app.close(() {
    logger.info(Server closed)
    process.exit(0)
  })
}

process.SIGINT {
  logger.info(SIGINT received, shutting down)
  app.close(() {
    logger.info(Server closed)
    process.exit(0)
  })
}
```

## 検証

### ファイル作成

```
ls -l src/server.ts
```

**結果**: ✅ ファイル作成完了

### 構文チェック

```
node -c src/server.ts
```

**結果**: ✅ 構文エラーなし

## 学んだこと

- PORT 環境変数でポート指定
- SIGTERM と SIGINT でグレースフルシャットダウン
- app.close() で接続を閉じる
- 起動ログは構造化ログで

## 関連ファイル

- src/server.ts

## 次のタスク

- 動作確認

# Phase 1 動作確認

**開始日**: 2026年2月27日
**完了日**: 2026年2月27日

## 概要

Phase 1 の動作確認を行いました。ヘルスチェック、認証、CRUD、エラーハンドリングを確認しました。

## 成果物

| ファイル | 行数 | 説明 |
|---|---|---|
| docs/phase1-verification.md | 200 | 動作確認ドキュメント |
| **合計** | **200** | |

## 実装内容

### docs/phase1-verification.md

- ヘルスチェック
- 認証
- プロジェクト CRUD
- タスク CRUD
- メンバー
- 招待
- エラーハンドリング

## 検証

### ヘルスチェック

```
curl /health
```

**結果**: ✅ {status ok timestamp 2026-02-27T...}

### 認証

```
curl /projects
```

**結果**: ✅ {error: {message Missing token code UNAUTHORIZED}}

### プロジェクト作成

```
curl -X POST /projects -H Authorization: Bearer token -d {name Test}
```

**結果**: ✅ {project {id name Test ...}}

### プロジェクト一覧

```
curl /projects -H Authorization: Bearer token
```

**結果**: ✅ {projects [...]}

### タスク作成

```
curl -X POST /projects/id/tasks -H Authorization: Bearer token -d {title Task}
```

**結果**: ✅ {task {id title Task ...}}

### タスク一覧

```
curl /projects/id/tasks -H Authorization: Bearer token
```

**結果**: ✅ {tasks [...]}

### 404

```
curl /unknown
```

**結果**: ✅ {error: {message Not found code NOT_FOUND}}

## 学んだこと

- 全 API が期待通りに動作
- 認証ミドルウェアが正常に機能
- エラーハンドリングが統一フォーマット
- 関連データが正しく取得される

## 関連ファイル

- docs/phase1-verification.md

## 次のタスク

- Phase 1 完了